

```

import hashlib.
from the distutils.dir_util import copy_tree.
from the collections, import OrderedDict.
import the OS.
import logger1 as log1.

```

The following code reads the Sync1.ini file:

```

def ConfRead():
    config = configparser.ConfigParser()
    config.read("Sync1.ini")
    return (dict(config.items("Para")))

```

Shown below are some of the variables obtained from the Sync.ini file:

```

All_Config = ConfRead()
Freq = int(All_Config.get("freq"))*60
Total_time = int(All_Config.get("total_time"))*60
repeat = int(Total_time/Freq)

```

The following function *md5* is used to calculate the hash of the file. If you modify a file, then the name remains the same but the hash gets changed.

```

def md5(fname,size=4096):
    hash_md5 = hashlib.md5()
    with open(fname, "rb") as f:
        for chunk in iter(lambda: f.read(size), b''):
            hash_md5.update(chunk)
    return hash_md5.hexdigest()

```

The following function copies the whole directory with intermediaries:

```

def CopyDir(from1, to):
    copy_tree(from1, to)

```

The following function just copies one file to the destination location:

```

def CopyFiles(file, path_to_copy):
    shutil.copy(file,path_to_copy)

```

The following function creates a dictionary, which contains the file names with the hash of the files. The function takes the source location and makes a dictionary of all the files present:

```

def OriginalFiles():
    drive = All_Config.get("from")
    Files_Dict = OrderedDict()
    print (drive)
    for root, dir, files in os.walk(drive, topdown=True):
        for file in files:

```

```

            file = file.lower()
            file_path = root+'\\'+file
            try:
                hash1 = md5(file_path,size=4096)
                #modification_time = int(os.path.
getmtime(file_path))
                rel_path = file_path.strip(drive)
                Files_Dict[(hash1,rel_path)]= file_path
            except Exception as e :
                log1.logger.error('Error Original files:
{0}'.format(e))

        return Files_Dict

```

The following function creates a dictionary that contains file names with a hash of the files. The function takes the destination location and gets all present files and makes a dictionary. If the root folder is not present, then it calls the *CopyDir* function.

```

def Destination():
    Files_Dict = OrderedDict()
    from1 = All_Config.get("from")
    to= All_Config.get("to")
    dir1= from1.rsplit("\\",1)[1]
    drive = to+dir1
    #print (drive)
    try:
        if os.path.isdir(drive):
            for root, dir, files in os.walk(drive,
topdown=True):
                for file in files:

                    file = file.lower()
                    file_path = root+'\\'+file
                    try:
                        hash1 = md5(file_path,size=4096)
                        #modification_time = int(os.path.
getmtime(file_path))
                        rel_path = file_path.strip(drive)
                        Files_Dict[(hash1,rel_path)]= file_
path

                    except Exception as e :
                        log1.logger.error('Error
Destination for loop: {0}'.format(e))
                        return Files_Dict
                    else :
                        CopyDir(from1,drive)
                        log1.logger.info('Full folder: {0} copied'.
format(from1))
                        return None
            except Exception as e :
                log1.logger.error('Error Destination: {0}'.format(e))

```